

多元线性回归中的分类变量处理

01 数据观察

- 数据集特征与可视化分析，探索数据分布规律
- 数值型变量(Numerical)：
 - R&D Spend(研发支出)/Administration(管理支出)
 - Marketing Spend(市场支出)/Profit(营收)
- 分类变量(Categorical)：
 - 由离散文本标签构成，无内在数值大小关系
 - 如 State(所在州/城市)：New York, California 等

导入数据集

```
[2]: dataset = pd.read_csv('50_Startups.csv')
dataset.head()
```

	R&D Spend-X1	Administration-X2	Marketing Spend-X3	State-X4	Profit-Y
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

```
[3]: X = dataset.iloc[:, 0:4].values
Y = dataset.iloc[:, 4].values
print(X[:10])
print(Y)
```

```
[[165349.2 136897.8 471784.1 'New York']
 [162597.7 151377.59 443898.53 'California']
 [153441.51 101145.55 407934.54 'Florida']
 [144372.41 118671.85 383199.62 'New York']
 [142107.34 91391.77 366168.42 'Florida']
 [131876.9 99814.71 362861.36 'New York']
 [134615.46 147198.87 127716.82 'California']
 [130298.13 145530.06 323876.68 'Florida']
 [120542.52 148718.95 311613.29 'New York']
 [123334.88 108679.17 304981.62 'California']]
[192261.83 191792.06 191050.39 182901.99 166187.94 156991.12 156122.51
 155752.6 152211.77 149759.96 146121.95 144259.4 141585.52 134307.35
 132602.65 129917.04 126992.93 125370.37 124266.9 122776.86 118474.03
 111313.02 110352.25 108733.99 108552.04 107404.34 105733.54 105008.31
 103282.38 101004.64 99937.59 97483.56 97427.84 96778.92 96712.8
 96479.51 90708.19 89949.14 81229.06 81005.76 78239.91 77798.83
 71498.49 69758.98 65200.33 64926.08 49490.75 42559.73 35673.41
 14681.4 ]
```



One-Hot 编码

One-Hot 编码核心原理

One-Hot编码(独热编码)的核心思想:
为每个类别特征创建独立的二进制列。
当样本属于某个类别时, 对应列标记为1,
其余所有类别对应的列均标记为0。

编码映射示例:

New York → [1, 0, 0]

California → [0, 1, 0]

Florida → [0, 0, 1]

解决的核心问题:

将无序离散类别转化为模型可处理的数值,
有效避免引入错误的"大小/顺序"关系。

```
[10]: array([[165349.2, 136897.8, 471784.1, 2],
          [162597.7, 151377.59, 443898.53, 0],
          [153441.51, 101145.55, 407934.54, 1],
          [144372.41, 118671.85, 383199.62, 2],
          [142107.34, 91391.77, 366168.42, 1],
          [131876.9, 99814.71, 362861.36, 2],
          [134615.46, 147198.87, 127716.82, 0],
          [130298.13, 145530.06, 323876.68, 1],
          [120542.52, 148718.95, 311613.29, 2],
          [123334.88, 108679.17, 304981.62, 0]], dtype=object)

[11]: ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3]), (remainder='passthrough')],
                             X = np.array(ct.fit_transform(X))
      #***由于现在的onhotencoder会默认对整个数据集进行处理, 并且没有categorical_features, 所以利用ColumnTransform
      #***其实也可以通过pandas中的get_dummies()来进行onehot编码***

[12]: X

[12]: array([[0.0, 0.0, 1.0, 165349.2, 136897.8, 471784.1],
          [1.0, 0.0, 0.0, 162597.7, 151377.59, 443898.53],
          [0.0, 1.0, 0.0, 153441.51, 101145.55, 407934.54],
          [0.0, 0.0, 1.0, 144372.41, 118671.85, 383199.62],
          [0.0, 1.0, 0.0, 142107.34, 91391.77, 366168.42],
          [0.0, 0.0, 1.0, 131876.9, 99814.71, 362861.36],
          [1.0, 0.0, 0.0, 134615.46, 147198.87, 127716.82],
          [0.0, 1.0, 0.0, 130298.13, 145530.06, 323876.68],
          [0.0, 0.0, 1.0, 120542.52, 148718.95, 311613.29],
          [1.0, 0.0, 0.0, 123334.88, 108679.17, 304981.62],
          [0.0, 1.0, 0.0, 101913.08, 110594.11, 229160.95],
          [1.0, 0.0, 0.0, 100671.96, 91790.61, 249744.55],
          [0.0, 1.0, 0.0, 93063.75, 127320.38, 249839.44],
          [1.0, 0.0, 0.0, 91992.39, 135495.07, 252664.93],
          [0.0, 1.0, 0.0, 119943.24, 156547.42, 256512.92],
          [0.0, 0.0, 1.0, 114523.61, 122616.84, 261776.23],
          [1.0, 0.0, 0.0, 78013.11, 121597.55, 264346.06],
          [0.0, 0.0, 1.0, 94657.16, 145077.58, 282574.31],
          [0.0, 1.0, 0.0, 91749.16, 114175.79, 294919.57],
          [0.0, 0.0, 1.0, 86419.7, 153514.11, 0.0],
          [1.0, 0.0, 0.0, 76253.06, 113867.3, 298064.47],
          [0.0, 0.0, 1.0, 78389.47, 153773.43, 299737.29],
          [0.0, 1.0, 0.0, 73994.56, 122782.75, 303319.26],
          [0.0, 1.0, 0.0, 67577.57, 105751.07, 304760.77]]
```



模型评估 — R^2 决定系数

R² 决定系数详解

```
[21]: R2 = regressor.score(X,Y)
      R2

[21]: 0.9490884301964865

[22]: bi = regressor.coef_ # 系数 (权重)
      bi

[22]: array([0.77884104, 0.0293919 , 0.03471025])
```

R²(决定系数)的含义:
衡量模型对因变量变异的解释程度

取值范围: 0 ~ 1

R²=1: 完美拟合, 解释了100%的变异

R²=0: 模型完全无效

R²越接近1, 拟合效果越好

Scikit-learn 用法:

`regressor.score(X_test, y_test)`

本次模型 R² ≈ 94.9%

模型能解释约94.9%的营收变化, 预测效果优秀



系数与截距分析

获取模型参数与结果解读

系数(Coefficients) `.coef_`

返回数组，每个元素对应自变量的系数

截距(Intercept) `.intercept_`

返回标量值，即回归方程的常数项

输出结果：

系数 `b1` = [0.7788, 0.0294, 0.0347]

截距 `b0` = 42989.01

预测模型公式：

$$\text{Profit} = 0.7788 \times \text{R\&D} + 0.0294 \times \text{Admin} \\ + 0.0347 \times \text{Marketing} + 42989.01$$

核心结论：

研发投入 (R&D) 系数0.7788 → 最大驱动因素

管理费用0.0294/营销0.0347 → 影响较弱



分类变量对回归的影响



05 分类变量对回归的影响

- 为什么必须对分类变量进行特殊处理？
 - 1. 模型无法处理非数值
 - 线性回归仅支持数值输入，直接传入字符串会报错
 - 2. 引入错误的序关系
 - 简单用LabelEncoder编码(1, 2, 3)，模型会错误认为类别间存在"大于"关系
 - 3. 遗漏重要特征信息
 - 分类变量蕴含显著的群体差异信息，忽略会降低模型精度
- 核心结论：必须对分类变量进行标准化处理
 - (如One-Hot独热编码)，才能构建准确的预测模型

可视化分析关键洞察

- 1. 研发投入是营收的核心驱动因素
- 三个城市回归线均呈强正向线性趋势，斜率高度一致
- 2. 管理费用与营收无显著线性关联
- 三个城市回归线斜率均接近0
- 3. 营销投入对营收有正向拉动
- 相关性强度弱于研发投入
- 4. 城市对营收直接影响有限
- 核心差异来自各公司的研发、营销投入结构
- 而非地域因素



➤ Label Encoder 的合理性分析 ◀

06 LabelEncoder的合理性分析

强烈不推荐的做法：

直接用LabelEncoder编码后输入回归模型

核心缺陷：强加"人为顺序"

模型会错误理解为连续数值，假设类别间存在线性关系

例如 $2 > 1 > 0$ ，但城市名无大小之分

One-Hot编码的核心优势：

- ① 消除虚假序关系 → 类别转为"正交并列"关系
- ② 特征表达更精准 → 稀疏矩阵，每行仅1个1

工程应用总结：

LabelEncoder仅适用于分类任务的目标变量(y)

回归模型的无序分类自变量(X)：

One-Hot是唯一标准解

总 结

① 识别与可视化

识别分类变量，通过可视化了解与因变量的潜在关系

② 正确编码方法

优先使用One-Hot编码，避免LabelEncoder引入错误序关系

③ 模型拟合评估

使用 R^2 决定系数，客观量化模型的拟合优度

④ 深度结果解读

分析系数与截距，定量理解每个自变量的影响方向与程度

⑤ 核心处理原则

分类变量必须经数值化处理后，才能纳入多元线性回归模型

总结 — 核心流程

- 完整流程回顾：
- Step 1: 识别分类变量与数值型变量
- Step 2: One-Hot编码转换分类变量
- Step 3: 建立多元线性回归模型
- Step 4: 用 R^2 评估模型拟合效果
- Step 5: 分析系数与截距，解读特征影响
- Step 6: 验证分类变量处理的重要性
- 核心公式：
- $\text{Profit} = 0.7788 \times \text{R\&D} + 0.0294 \times \text{Admin}$
- $+ 0.0347 \times \text{Marketing} + 42989.01$
- $R^2 = 94.9\%$

分类变量处理核心流程

- 引言：从线性回归到分类变量挑战
- 回归模型基础： $y = \beta_0 + \beta_1 x_1 + \dots + \varepsilon$
- 线性回归是预测连续型因变量的经典工具
- 在标准模型假设中，自变量 x 是连续数值型变量
- 现实数据的挑战：分类变量 (Categorical Variables)
- 真实场景中充满非数值信息：性别(男/女)、城市(北京/上海)
- 这些标签无法直接代入回归公式进行数学计算
- 核心目标：将无序的分类信息有效转化为
- 回归模型可以理解的数值形式



THANKS FOR WATCHING



信管2302 钟京序
202321054056